

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.Bhavya Sri	Reg. No.:	192411249
Section / Batch:	Slot C	Date:	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Each job has a deadline and a profit, and only one job can be scheduled at a time. The objective is to schedule the jobs such that the **total profit is maximized** while ensuring that each selected job is completed before its deadline. Conflicts must be avoided, and the jobs should be scheduled in an optimal order. Use a **Greedy Strategy** to determine the best job schedule.

Code of Conduct

I **CH.Bhavya Sri/12411249**____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.Bhavya Sri_____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab
DAA PROGRAMMING

CHENNABOINA BHAVYA SRI
192411249RC

QUESTIONS

Q1
Job Scheduling with Deadlines
5 marks

1/1 answered

Q1 Job Scheduling with Deadlines5 marks

Each job has a deadline and a profit, and only one job can be scheduled at a time. The objective is to schedule jobs such that total profit is maximized while ensuring each job is completed before its deadline. Conflicts must be avoided, and optimal ordering is required. Use a Greedy strategy to determine the best schedule. Input Format: n = number of jobs deadline[] = deadlines profit[] = profits Sample Input: n = 4 deadline = 2 1 2 1 profit = 100 19 27 25 Expected Output: Max Profit = 127

Your Code

```
1 def solve():
2     try:
3         # Reading input
4         import sys
5         input_data = sys.stdin.read().split()
6         if not input_data:
7             return
8
9         n = int(input_data[0])
10        # The input format shows deadlines as a single string/sequence
11        # But based on logic, we need n deadlines and n profits.
12        # Let's parse the deadlines and profits from the provided stru
```

Input

4 2121
100 19 27 25

Output

Max Profit = 127

EXPLANATION

Job Scheduling with Deadlines is an optimization problem solved using the **Greedy Method**. In this problem, every job has a deadline and a profit, and each job requires one unit of time to complete. The main objective is to select and schedule the jobs in such a way that the **maximum possible profit** is obtained while making sure that every selected job is completed before its deadline. To solve the problem, we first sort all the jobs according to their profits in **descending order**. Then, starting with the job having the highest profit, we try to place it in the latest available time slot before its deadline. This helps to keep earlier slots available for other jobs. In the given example, J1 has the highest profit of 100 and deadline 2, so it is placed in the second slot. J3 has a profit of 27 and deadline 2, so it is placed in the first slot. The other jobs cannot be selected because all available slots are occupied. Thus, the selected jobs are **J3 and J1**, and the total maximum profit is $27 + 100 = 127$. This method is called greedy because it always makes the locally best choice by selecting the job with the highest profit at each step.